Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет Программная инженерия и компьютерная техника

Образовательная программа Компьютерные системы и технологии

Направление подготовки (специальность) 09.03.01 - Информатика и вычислительная техника

О Т Ч Е Т

О производственной, технологической практике

Тема задания: Разработка интерактивного тренажёра «Алгоритм Флёри: построение Эйлерова цикла в неориентированном графе» для системы дистанционного обучения ИТМО

Обучающийся: Нодири Хисравхон, P3431

Руководитель практики от профильной организации: Филькова Инна Александровна, заместитель декана факультета безопасности информационных технологий, Университет ИТМО

Руководитель практики от университета: Маркина Татьяна Анатольевна, старший преподаватель факультета программной инженерии и компьютерной техники, Университет ИТМО

Дата: 05.04.2026

г. Санкт-Петербург
2026

Содержание

Обозначения и сокращения **3**

Характеристика организации (места прохождения практики) **4**

Введение **5**

1 Согласование функциональных требований к тренажёру **6**

1.1 Содержание этапов практики 6

1.2 Выполнение этапа согласования требований 8

1.3 Теоретическая основа и конкретизация постановки задачи 8

2 Проектирование интерактивного тренажёра **9**

2.1 Проработка пользовательских сценариев и требований 9

2.2 Проектирование архитектуры и логики взаимодействия 10

2.3 Уточнение модели данных и сценариев работы 11

3 Разработка тренажёра и интеграция с LMS **12**

3.1 Последовательность реализации серверной части 12

3.2 Разработка клиентской части 13

3.3 Разработка механизма оценивания и подготовка к интеграции 14

4 Тестирование и устранение ошибок **15**

4.1 Подготовка сценариев проверки 15

4.2 Выявленные проблемы и выполненные исправления 16

Заключение **17**

Список использованных источников **18**

Приложение А. Материалы этапов согласования требований и проектирования . **19**

Приложение Б. Материалы этапа разработки **20**

Приложение В. Материалы этапа интеграции с LMS **22**

Приложение Г. Материалы этапа тестирования **23**

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

API – application programming interface, программный интерфейс приложения.

CSS – cascading style sheets, каскадные таблицы стилей.

HTML – hypertext markup language, язык гипертекстовой разметки.

ITMO – Университет ИТМО.

JSON – javascript object notation, формат обмена структурированными данными.

LMS – learning management system, система дистанционного обучения.

UI – user interface, пользовательский интерфейс.

ХАРАКТЕРИСТИКА ОРГАНИЗАЦИИ (МЕСТА ПРОХОЖДЕНИЯ ПРАКТИКИ)

Местом прохождения практики являлся Университет ИТМО, а предметной средой работ выступала система дистанционного обучения, в которой размещаются интерактивные задания для студентов. Такая среда предъявляет к программным средствам одновременно методические и технические требования: интерфейс должен быть понятным для обучающихся, алгоритмическая логика – корректной, а развёртывание – воспроизводимым в изолированной контейнерной инфраструктуре.

Практическая работа выполнялась в контексте разработки учебного веб-приложения, предназначенного для закрепления темы эйлеровых циклов и алгоритма Флёри. Целевой аудиторией тренажёра являются студенты, изучающие дискретную математику и теорию графов, а также преподаватели, которым необходим инструмент для выдачи самостоятельных заданий, автоматизированной проверки и демонстрации корректного построения эйлерова цикла.

Особенностью места прохождения практики является ориентация на дистанционный формат. Это означает, что программный продукт должен одинаково поддерживать учебный сценарий, режим контролируемого тестирования, передачу результатов в внешнюю среду и запуск в стандартной инфраструктуре LMS. Подобные требования соответствуют целям производственной, технологической (проектно-технологической) практики, в рамках которой результатом работы является не только код, но и законченный цифровой артефакт, пригодный для эксплуатации в учебном процессе [1, 2].

ВВЕДЕНИЕ

Производственная, технологическая (проектно-технологическая) практика направлена на получение и закрепление навыков практической профессиональной деятельности, связанных с анализом требований, проектированием, программной реализацией и документированием программных средств [1]. Для ИТМО особенно актуальны разработки, которые могут быть интегрированы в цифровую образовательную среду и использоваться в дистанционном обучении.

Тема настоящей практики – разработка интерактивного тренажёра «Алгоритм Флёри: построение эйлерова цикла в неориентированном графе» для системы дистанционного обучения ИТМО. Выбор темы обусловлен двумя факторами. Во-первых, алгоритм Флёри является классическим алгоритмом теории графов и используется для объяснения условий существования и построения эйлеровых циклов [3, 4]. Во-вторых, для эффективного освоения этой темы студентам недостаточно статического текста: необходимы визуализация графа, пошаговое прохождение и проверка собственных действий.

Объектом практики является образовательное веб-приложение, предназначенное для работы с неориентированными графами. Предмет практики – методы проектирования и реализации интерактивного тренажёра, обеспечивающего генерацию эйлеровых графов, демонстрацию алгоритма Флёри и контроль усвоения материала.

Цель работы состояла в разработке интерактивного тренажёра, который можно использовать в системе дистанционного обучения ИТМО для изучения и проверки навыков построения эйлерова цикла в неориентированном графе.

Для достижения поставленной цели были решены следующие задачи:

1. проанализирована предметная область и сформулированы функциональные и нефункциональные требования к тренажёру;
2. спроектированы структура приложения, модель данных и пользовательские сценарии;
3. реализованы серверная и клиентская части приложения;
4. разработан режим контролируемого тестирования с оцениванием качества решения;
5. подготовлены материалы по развёртыванию и комплект отчётной документации в соответствии с требованиями методического пособия [1, 2].

В основной части отчёта подробно рассмотрено выполнение этапов 2–5 рабочего плана-графика: согласование функциональных требований, проектирование интерфейса и логики тренажёра, разработка и интеграция в LMS, тестирование и устранение ошибок. Для каждого этапа описаны не только полученные результаты, но и последовательность выполненных действий, принятые решения и выполненные доработки. В приложениях приведены материалы, подготовленные по итогам этих этапов.

1 Согласование функциональных требований к тренажёру

1.1 Содержание этапов практики

Работа в ходе практики была организована по утверждённому рабочему плану-графику. Для последующего анализа важно было зафиксировать не только конечный программный результат, но и логику прохождения этапов, поскольку именно она отражает процесс формирования требований, проектирования и программной реализации. Структура этапов приведена в таблице 1.

Таблица 1 – Этапы выполнения практики по рабочему плану-графику

№	Наименование этапа	Период	Содержание работ
1	Инструктаж обучающегося	1	Ознакомление с требованиями охраны труда, техники безопасности, пожарной безопасности и правилами внутреннего распорядка.
2	Обсуждение с куратором функциональных требований к интерактивному тренажёру по алгоритму Флёри	3	Определение формата визуализации графа, сценариев пошагового прохождения алгоритма, типов заданий и критериев оценки ответов.
3	Проектирование интерфейса и логики тренажёра	6	Разработка структуры экрана, элементов управления, схемы взаимодействия модулей и прототипа пользовательского сценария.
4	Разработка тренажёра и интеграция с LMS	13	Реализация серверной и клиентской частей, настройка режимов работы, параметризация запуска и подготовка конфигурации для среды дистанционного обучения.
5	Тестирование и устранение ошибок	13	Подготовка набора проверок, выполнение функционального тестирования, анализ граничных случаев и исправление обнаруженных недостатков.
6	Оформление отчётных документов в соответствии с требованиями	15	Подготовка отчёта, приложений и сопроводительных материалов по итогам практики.
7	Получение отзыва руководителя практики	7	Передача результатов практики руководителю и получение итогового отзыва с оценкой.

В основной части далее подробно рассматриваются этапы 2–5, поскольку именно на них выполнялась содержательная работа по созданию тренажёра. Этапы 1, 6 и 7 носят сопроводительный характер: они обеспечивают формальное начало практики, оформление результатов и итоговую оценку.

1.2 Выполнение этапа согласования требований

Работа над проектом началась не с написания кода, а с обсуждения задачи с куратором практики. На этом этапе требовалось перевести общую тему практики в набор конкретных требований к программному средству. Обсуждение было сосредоточено на том, каким должен быть тренажёр с точки зрения учебного сценария, какие действия должен выполнять пользователь и какие результаты должны фиксироваться системой.

В ходе согласования были последовательно рассмотрены следующие вопросы:

– как должен быть представлен граф на экране и какие действия пользователь должен выполнять вручную;

– должен ли тренажёр только демонстрировать готовое решение или также проверять действия обучающегося;

– какие режимы работы необходимы для учебного и контрольного использования;

– какие ограничения и критерии оценивания следует заложить в тестовый режим;

– каким образом подготовленный тренажёр будет запускаться в среде дистанционного обучения.

Первоначально задача могла быть сведена к обычной визуализации алгоритма Флёри, однако после обсуждения стало ясно, что этого недостаточно для целей практики. Было решено, что система должна поддерживать не один, а несколько сценариев использования: самостоятельное построение и редактирование графа, пошаговую демонстрацию алгоритма, автоматическое выполнение и отдельный режим контролируемого тестирования. Такое уточнение повлияло на все последующие этапы: уже на стадии проектирования потребовалось учитывать как обучающую, так и оценочную функции тренажёра.

Отдельным результатом этапа стало уточнение состава проверяемых действий пользователя. В тестовом режиме необходимо было оценивать не только факт построения эйлерова цикла, но и корректность выбора рёбер в соответствии с правилом Флёри. Это решение усложнило постановку задачи, поскольку потребовало реализовать анализ пользовательского ответа на уровне отдельных шагов, а не только на уровне конечного маршрута.

1.3 Теоретическая основа и конкретизация постановки задачи

После согласования общих требований была уточнена математическая основа тренажёра. Эйлеровым циклом называется замкнутый маршрут в графе, проходящий по каждому ребру ровно один раз. Для существования эйлерова цикла в неориентированном графе необходимо, чтобы граф был связным по вершинам ненулевой степени и степени всех его вершин были чётными [5, 6]. В отчёте это условие записывается следующим образом:

$$\forall v \in V \quad \deg(v) \equiv 0 \pmod 2, \tag{1}$$

где V – множество вершин графа, $\deg(v)$ – степень вершины v.

Далее была проанализирована специфика самого алгоритма Флёри. Он строит эйлеров цикл поэтапно: на каждом шаге из текущей вершины выбирается ребро, которое не является мостом в остаточном графе, если существует альтернативный выбор [3, 4]. Из этого следовало, что для корректного тренажёра требуется не только вычислять очередной шаг, но и объяснять пользователю, почему данный шаг допустим или вынужден.

По итогам этапа 2 задача была сформулирована в рабочем виде как разработка веб-приложения для системы дистанционного обучения ИТМО, которое:

- визуализирует неориентированный граф и ход алгоритма;
- позволяет пользователю самостоятельно задавать или генерировать эйлеров граф;
- поддерживает пошаговое и автоматическое выполнение алгоритма Флёри;
- содержит отдельный режим тестирования с автоматической оценкой ответа;
- может быть воспроизводимо запущено в стандартной среде LMS.

Именно в таком виде постановка задачи была передана на следующий этап, где требовалось уже не обсуждать требования, а проектировать конкретную структуру интерфейса и внутренней логики системы.

2 Проектирование интерактивного тренажёра

2.1 Проработка пользовательских сценариев и требований

После согласования общей постановки задачи работа была перенесена на уровень проектирования. На этом этапе требовалось превратить словесно сформулированные ожидания куратора в структуру экранов, набор элементов управления и последовательность пользовательских действий. Проектирование начиналось с выделения основных сценариев использования: редактирование графа, пошаговое прохождение алгоритма, автоматическая демонстрация и контролируемое тестирование.

Сначала был зафиксирован минимальный набор действий, который пользователь должен выполнять без переключения между разными страницами. Это решение было принципиальным: поскольку тренажёр предназначен для LMS, интерфейс должен оставаться компактным, а все основные операции должны быть доступны из одного экрана. В результате была выбрана двухпанельная композиция: слева размещается рабочая область с графом и результатами, справа – элементы управления, настройки и блок тестирования.

Следующим шагом стало уточнение функциональных требований к каждому сценарию. Сводка ключевых требований, сформированная на этапе проектирования, приведена в таблице 2.

Таблица 2 – Ключевые требования к тренажёру

Группа требований	**Содержание**
Работа с графом	Добавление и удаление вершин и рёбер, очистка графа, генерация случайного эйлерова графа, выбор стартовой вершины.
Учебный режим	Пошаговый запуск алгоритма, автоматическое выполнение, журнал действий, визуальная подсветка стартовой и текущей вершины, отображение пройденных рёбер и мостов.
Тестовый режим	Генерация задания, ввод цикла Эйлера для каждой вершины, ограничение времени и числа попыток, автоматическая проверка и выставление баллов.
Надёжность вычислений	Валидация входных данных, проверка условий эйлеровости, защита от некорректных имён вершин и несуществующих рёбер.
Интеграция	Запуск в контейнерной инфраструктуре, настройка через переменные окружения, подготовка конфигурации для локальной разработки и production-сборки.
Педагогическая пригодность	Простая визуальная схема, минимальное число действий до получения результата, наглядная обратная связь при ошибках пользователя.

Таблица требований использовалась не как итоговая справка, а как рабочий инструмент проектирования. По ней последовательно проверялось, не потеряна ли какая-либо функция при переходе от одного проектного решения к другому. Например, требование по отображению мостов повлияло на состав визуальных состояний рёбер, а требование по контролируемому тестированию – на появление отдельного механизма анализа ответов и системы начисления баллов.

2.2 Проектирование архитектуры и логики взаимодействия

После фиксации пользовательских сценариев был выполнен переход к проектированию внутренней структуры системы. На этом шаге нужно было определить, какие части логики следует реализовать на сервере, какие – в браузере, и как организовать обмен данными между ними. Ключевым решением стало разделение системы на клиентский уровень, серверное Flask-приложение и отдельный алгоритмический слой. Такое разделение позволило не смешивать визуализацию с вычислительной логикой и обеспечило повторное использо-

вание алгоритмических функций в разных режимах работы.

Структура разработанного тренажёра представлена на рисунке 1.

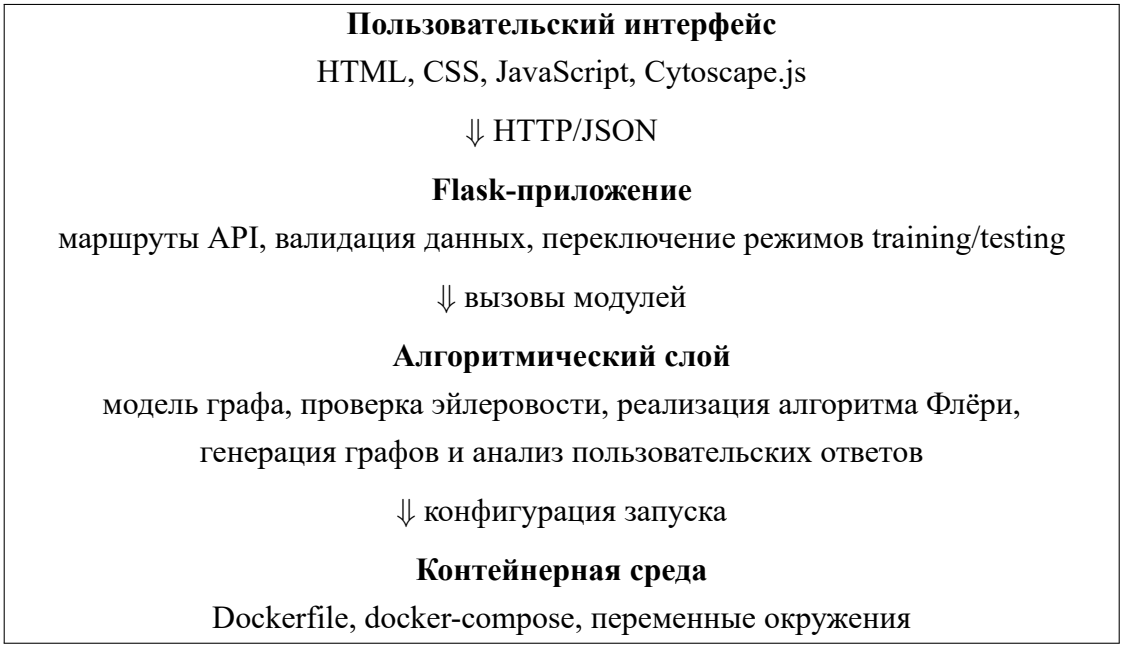


Рис. 1 – Структура интерактивного тренажёра

В процессе проектирования были выделены следующие основные сущности серверной части:

- модель графа, включающая список вершин и рёбер, операции модификации и базовые методы анализа;
- модуль реализации алгоритма Флёри, отвечающий за инициализацию, пошаговое выполнение и формирование текущего состояния;
- функции анализа пользовательского решения, проверяющие корректность пути и соответствие правилу Флёри;
- слой HTTP-маршрутов, предоставляющий интерфейс для клиентского приложения.

Выбор такой декомпозиции был связан с практическими соображениями. Если бы логика проверки и алгоритм Флёри находились только в клиентской части, было бы труднее обеспечивать единообразие поведения в учебном и тестовом режимах. Перенос вычислительной части на сервер позволил централизовать валидацию, упростить проверку ответов и уменьшить зависимость от состояния пользовательского интерфейса. Подобная архитектура соответствует типовой практике разработки веб-приложений на Python и Flask [7, 8].

2.3 Уточнение модели данных и сценариев работы

Отдельной задачей этапа проектирования стало определение формата представления графа. Необходимо было выбрать такую модель данных, которая, с одной стороны, позволяла бы выполнять алгоритмическую обработку на сервере, а с другой – без лишних преобразований передавалась бы в браузер для визуализации. По этой причине была выбрана

простая структура: список идентификаторов вершин и список рёбер вида «источник – цель». Такой формат удобно сериализуется в JSON и напрямую передаётся в клиентскую часть, где используется библиотека Cytoscape.js [9].

После выбора модели данных был уточнён состав сценариев работы системы.

Первый сценарий – учебный. Пользователь самостоятельно задаёт или генерирует граф, выбирает стартовую вершину и запускает алгоритм по шагам или автоматически. На этапе проектирования для этого сценария специально были заложены журнал действий, таблица маршрута и визуальная подсветка состояния графа. Эти элементы появились не как декоративные части интерфейса, а как ответ на требование сделать процесс прохождения алгоритма объяснимым.

Второй сценарий – контрольный. Здесь проектирование было сложнее, поскольку требовалось совместить автоматическую проверку с понятной для пользователя обратной связью. Было решено, что система будет генерировать граф, принимать ответы по каждой стартовой вершине и после проверки возвращать не только итоговый балл, но и частные показатели качества решения: число корректно использованных рёбер, число замкнутых циклов и число нарушений правила Флёри. Наличие двух сценариев было зафиксировано как обязательное проектное решение, поскольку именно оно делало тренажёр одновременно учебным и контрольным инструментом.

3 Разработка тренажёра и интеграция с LMS

3.1 Последовательность реализации серверной части

Этап разработки был начат с серверной части, поскольку именно здесь сосредоточена алгоритмическая логика, определяющая корректность всего тренажёра. Реализация велась поэтапно: сначала были созданы базовые операции над графом, затем проверка условий эйлеровости, после этого – механизм пошагового выполнения алгоритма, и только затем поверх этих сущностей был организован слой HTTP-маршрутов.

Серверная часть реализована на языке Python с использованием микрофреймворка Flask [7, 8]. В модуле `app.py` инициализируется приложение, настраиваются режимы работы и описываются HTTP-маршруты. Выбор Flask был обусловлен необходимостью быстро построить компактное веб-приложение с явным разделением маршрутов, входных данных и режимов работы. Переменная окружения `APP_MODE` определяет, запускается ли приложение в режиме обучения или тестирования, а дополнительные параметры задают число попыток, максимальный балл, временной лимит и диапазон размера графа.

Алгоритмическая логика вынесена в модуль `algorithm.py`. Ключевым элементом

является класс `Graph`, в котором реализованы методы добавления и удаления вершин и рёбер, вычисления степени вершины, проверки достижимости и клонирования графа. Отдельная функция `check_eulerian_graph` проверяет выполнение условий эйлеровости, сформулированных в (1), а также связность графа по вершинам ненулевой степени. Такой порядок разработки позволил сначала получить устойчивое ядро для операций над графом, а затем строить поверх него прикладную логику тренажёра.

После этого был реализован класс `FleuryAlgorithm`, инкапсулирующий состояние пошагового выполнения. Он хранит исходный граф, его рабочую копию, стартовую вершину, построенный путь и список уже использованных рёбер. Основная трудность на данном шаге заключалась в том, чтобы корректно различать вынужденные переходы и случаи, когда пользователь должен избегать выбора моста. Для решения этой задачи была реализована отдельная проверка мостов в остаточном графе, а состояние шага было расширено служебной информацией, необходимой для последующей визуализации и объяснения выбора ребра пользователю.

Заметным элементом этапа реализации стала функция генерации случайного эйлерова графа. Изначально рассматривался вариант случайного добавления рёбер, однако он приводил к большому числу некорректных или слишком простых графов. Поэтому был выбран более устойчивый подход: сначала строится простой цикл по всем вершинам, а затем случайным образом «переключаются» дополнительные циклы. Такая операция сохраняет чётность степеней вершин и при дополнительной проверке позволяет получать более разнообразные, но корректные эйлеровы графы.

3.2 Разработка клиентской части

После стабилизации серверной логики началась реализация клиентской части. Работа над интерфейсом велась итеративно: сначала был собран минимальный рабочий экран, после чего по результатам внутренней проверки уточнялись расположение элементов, состав кнопок управления и способ отображения промежуточных состояний алгоритма.

Клиентская часть состоит из HTML-шаблона, CSS-стилей и нескольких JavaScript-модулей. Разметка страницы задаёт двухпанельную структуру: слева располагается область графа, журнал и таблица текущего маршрута, справа – редактор графа, элементы управления алгоритмом, блок тестирования, легенда и описание. Такая структура появилась как прямое продолжение этапа проектирования: она позволила сохранить на одном экране и область визуализации, и управляющие элементы, и результаты работы алгоритма.

Визуализация графа выполняется библиотекой Cytoscape.js [9]. Для вершин и рёбер были последовательно настроены стили, отражающие их текущее состояние: стартовая вершина, текущая вершина, вершины маршрута, уже пройденные рёбра, мосты и повторно использованные рёбра. Это решение было принципиальным для учебного режима, поскольку позволяло превратить сухой вычислительный результат в наглядную пошаговую демонстрацию.

Для снижения связности клиентский код был разделён на модули. Модуль `training.js` отвечает за работу элементов редактора графа и учебного режима: через него обрабатываются добавление и удаление вершин и рёбер, очистка графа, генерация случайного примера и запуск алгоритма. Модуль `algorithm.js` реализует взаимодействие с сервером для инициализации, выполнения шага и автоматического прохождения алгоритма, а также обновляет журнал и таблицу маршрута. Модуль `testing.js` отвечает за сценарий контролируемого тестирования и формирует поля ввода для ответов по каждой стартовой вершине. Такое разделение облегчило последовательную доработку отдельных сценариев без переписывания всего интерфейса.

3.3 Разработка механизма оценивания и подготовка к интеграции

В рамках этапа разработки отдельно была реализована система оценивания тестового режима. Итоговая относительная оценка попытки вычисляется по формуле

$$R = 0{,}60 \cdot r_{\text{cov}} + 0{,}15 \cdot r_{\text{cycle}} + 0{,}25 \cdot r_{\text{fleury}}, \tag{2}$$

где r_{cov} – доля корректно использованных рёбер, r_{cycle} – доля правильно замкнутых циклов, r_{fleury} – доля шагов, удовлетворяющих правилу Флёри.

Весовые коэффициенты в формуле (2) были подобраны в соответствии с согласованными критериями оценивания. Наибольший вклад в итоговую оценку даёт корректное прохождение по всем рёбрам, поскольку именно это отражает достижение основной учебной цели. Отдельно учитываются факт замыкания цикла и соблюдение правила выбора немостового ребра. После вычисления относительной оценки она преобразуется в дискретный балл, используемый системой дистанционного обучения.

Параллельно с реализацией логики оценивания велась подготовка приложения к интеграции в LMS. Для обеспечения воспроизводимости запуска были подготовлены файлы `Dockerfile`, `dev.yml`, `docker-compose.yml` и `spec.yml`. Конфигурация локальной разработки публикует сервис на порт `3000` и включает режим отладки, тогда как production-конфигурация задаёт параметры контейнера, лимиты ресурсов, сетевое окружение и набор переменных окружения для LMS [13]. Это позволило завершить этап разработки не только рабочим кодом, но и подготовленной схемой запуска в учебной инфраструктуре.

4 Тестирование и устранение ошибок

4.1 Подготовка сценариев проверки

После завершения основной реализации был сформирован набор функциональных проверок. Работа на этом этапе велась от пользовательских сценариев к конкретным тестовым случаям: сначала были выделены ключевые действия пользователя, затем для каждого из них были определены ожидаемые результаты и возможные граничные состояния. Сводка подготовленных сценариев приведена в таблице 3.

Таблица 3 – Основные сценарии функциональной проверки

Проверяемый сценарий	**Ожидаемый результат**
Создание и редактирование графа	Вершины и рёбра корректно добавляются, удаляются и отображаются в интерфейсе.
Проверка неэйлерова графа	Алгоритм не запускается, пользователю возвращается объясняющее сообщение о причине отказа.
Пошаговое выполнение алгоритма	На каждом шаге корректно обновляются журнал, таблица маршрута и визуальное состояние графа.
Автоматический запуск	Алгоритм завершает построение эйлерова цикла без потери согласованности состояния интерфейса.
Режим тестирования	Ответы пользователя валидируются, ограничения по времени и попыткам соблюдаются, итоговый балл вычисляется корректно.
Повторный запуск в контейнере	Приложение поднимается через конфигурацию локальной разработки и production-описание без ручной правки кода.

Подготовленный набор проверок использовался не только для подтверждения корректности работы, но и как средство поиска мест, требующих доработки. Особое внимание было уделено граничным случаям: пустому графу, графу с нечётными степенями вершин, повторному прохождению по одному и тому же ребру, вводу несуществующих вершин, нарушению структуры ответа и завершению теста по истечении времени.

4.2 Выявленные проблемы и выполненные исправления

Тестирование проводилось итеративно. После каждого найденного недостатка выполнялось исправление, затем соответствующий сценарий запускался повторно, а смежные функции дополнительно проверялись на отсутствие регрессий. Такой подход оказался особенно важным для случаев, в которых клиентская и серверная части влияют друг на друга.

По результатам тестирования были выполнены следующие группы исправлений:

- усилена валидация входных данных, чтобы алгоритм не запускался для пустых, неэйлеровых и некорректно заданных графов;
- уточнено обновление клиентского состояния после каждого шага алгоритма, чтобы журнал, таблица маршрута и визуальное состояние графа оставались согласованными;
- доработана логика выбора рёбер в пошаговом режиме, чтобы вынужденные переходы и мосты корректно определялись и отражались в интерфейсе;
- проверены ограничения тестового режима по времени, числу попыток и формату ответов пользователя;
- подтверждена воспроизводимость контейнерного запуска без ручного изменения исходного кода и конфигурации.

После внесения исправлений подготовленные сценарии были выполнены повторно. Повторная проверка показала, что ключевые пользовательские сценарии работают стабильно, а граничные случаи обрабатываются явно, без скрытых ошибок и неоднозначных состояний интерфейса. Тем самым этап 5 завершился не только фиксацией качества результата, но и реальным циклом поиска, анализа и устранения обнаруженных проблем.

ЗАКЛЮЧЕНИЕ

В ходе производственной, технологической (проектно-технологической) практики была выполнена разработка интерактивного тренажёра «Алгоритм Флёри: построение эйлерова цикла в неориентированном графе» для системы дистанционного обучения ИТМО.

В ходе этапа согласования требований были уточнены учебные и технические ожидания к тренажёру, определены ключевые сценарии использования и зафиксированы критерии оценивания ответов пользователя. На этапе проектирования были разработаны структура интерфейса, архитектура взаимодействия клиентской, серверной и алгоритмической частей, а также модель представления графа и пользовательских действий. На этапе разработки были последовательно реализованы серверная логика, клиентский интерфейс, система оценивания и контейнерная конфигурация для интеграции в LMS. На этапе тестирования был сформирован набор сценариев проверки, выявлены проблемные случаи и выполнены исправления, обеспечившие устойчивую работу системы.

Практический результат работы представляет собой самодостаточное учебное веб-приложение, пригодное для демонстрации алгоритма Флёри, самостоятельной тренировки и текущего контроля знаний. После завершения технических этапов были также подготовлены отчётные материалы в соответствии с требованиями методического пособия. Поставленная цель достигнута, задачи практики выполнены в полном объёме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Маркина Т. А., Пенской А. В., Штенников Д. Г., Авксентьева Е. Ю., Ильина А. Г. Производственная практика магистрантов: организация и проведение. СПб.: Университет ИТМО, 2020. 50 с.
2. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. М.: Стандартинформ, 2017. 32 с.
3. Wilson R. J. Introduction to Graph Theory. 5th ed. Harlow: Pearson, 2010. 648 p.
4. West D. B. Introduction to Graph Theory. 2nd ed. Upper Saddle River: Prentice Hall, 2001. 588 p.
5. Bondy J. A., Murty U. S. R. Graph Theory. New York: Springer, 2008. 651 p.
6. Diestel R. Graph Theory. 5th ed. Berlin: Springer, 2017. 428 p.
7. Python Software Foundation. Python 3 Documentation [Электронный ресурс]. Режим доступа: https://docs.python.org/3/, свободный (дата обращения: 04.04.2026).
8. Pallets. Flask Documentation [Электронный ресурс]. Режим доступа: https://flask.palletsprojects.com/, свободный (дата обращения: 04.04.2026).
9. Cytoscape.js Documentation [Электронный ресурс]. Режим доступа: https://js.cytoscape.org/, свободный (дата обращения: 04.04.2026).
10. WHATWG. HTML Living Standard [Электронный ресурс]. Режим доступа: https://html.spec.whatwg.org/, свободный (дата обращения: 04.04.2026).
11. W3C. CSS Snapshot [Электронный ресурс]. Режим доступа: https://www.w3.org/TR/css/, свободный (дата обращения: 04.04.2026).
12. ECMA International. ECMAScript Language Specification [Электронный ресурс]. Режим доступа: https://tc39.es/ecma262/, свободный (дата обращения: 04.04.2026).
13. Docker Docs. Compose file reference [Электронный ресурс]. Режим доступа: https://docs.docker.com/reference/compose-file/, свободный (дата обращения: 04.04.2026).

Приложение А

Материалы этапов согласования требований и проектирования

В таблице 4 приведены основные пользовательские сценарии, сформированные по результатам этапа анализа требований.

Таблица 4 – Основные пользовательские сценарии тренажёра

Сценарий	**Краткое описание**
Редактирование графа	Пользователь создаёт вершины и рёбра, удаляет ошибочные элементы, очищает рабочее поле или генерирует случайный пример.
Пошаговое обучение	Пользователь выбирает стартовую вершину и последовательно запускает шаги алгоритма, наблюдая журнал и маршрут.
Автоматическая демонстрация	Система полностью выполняет алгоритм и показывает готовый эйлеров цикл.
Контролируемое тестирование	Пользователь вводит эйлеровы циклы, а система проверяет корректность рёбер, замыкание циклов и соблюдение правила Флёри.
Публикация результата в LMS	После завершения теста система возвращает итоговый балл и формирует данные для внешней фиксации результата.

На рисунке 2 приведена укрупнённая схема взаимодействия компонентов, использованная на этапе проектирования.

Пользователь → интерфейс браузера → HTTP API →
модуль алгоритма и проверки → результат и визуальная обратная связь

Рис. 2 – Укрупнённая схема взаимодействия компонентов

Приложение Б

Материалы этапа разработки

Ниже приведён сокращённый фрагмент функции генерации случайного эйлерова графа, отражающий принцип сохранения чётности степеней вершин.

Listing 1: Фрагмент функции генерации эйлерова графа

```
def generate_eulerian_graph(min_nodes=6, max_nodes=9):
    node_count = random.randint(min_nodes, max_nodes)
    nodes = [chr(ord('A') + i) for i in range(node_count)]

    edge_keys = set()
    for i in range(node_count):
        source = nodes[i]
        target = nodes[(i + 1) % node_count]
        edge_keys.add(Graph.edge_key(source, target))

    target_ops = random.randint(max(2, node_count // 2), max(4, node_count * 2))
    attempts_left = target_ops * 8

    while target_ops > 0 and attempts_left > 0:
        attempts_left -= 1
        cycle_len = random.randint(3, node_count)
        cycle_nodes = random.sample(nodes, cycle_len)
        cycle_edges = [
            Graph.edge_key(cycle_nodes[i], cycle_nodes[(i + 1) % cycle_len])
            for i in range(cycle_len)
        ]
```

Следующий фрагмент отражает центральную часть реализации алгоритма Флёри: на каждом шаге из текущей вершины выбирается допустимое ребро, причём при наличии альтернативы предпочтение отдаётся ребру, не являющемуся мостом в остаточном графе.

Listing 2: Фрагмент пошагового выполнения алгоритма Флёри

```
def step(self) -> Dict:
    if self.finished:
        return self.get_state()

    neighbors = self.graph.get_neighbors(self.current_node)

    if not neighbors:
        self.finished = True
        if self.graph.edge_count() == 0 and len(self.path) > 1 and self.path[0] == self.path[-1]:
```

```
            self.valid = True
            self.message = 'Эйлеров цикл построен.'
        else:
            self.valid = False
            self.message = 'Цикл не построен: закончились доступные ребра.'
        return self.get_state()

    non_bridge_neighbors = [
        neighbor for neighbor in neighbors
        if not self._is_bridge(self.current_node, neighbor)
    ]

    forced = len(neighbors) == 1 or len(non_bridge_neighbors) == 0
    next_node = sorted(neighbors)[0] if forced else sorted(non_bridge_neighbors)[0]
    is_bridge = self._is_bridge(self.current_node, next_node)

    self.graph.remove_edge(self.current_node, next_node)
```

Следующий фрагмент иллюстрирует вычисление качества пользовательского решения в тестовом режиме.

Listing 3: Фрагмент вычисления оценки решения

```
SCORE_WEIGHTS = {
    'edge_coverage': 0.60,
    'cycle_complete': 0.15,
    'fleury_rule': 0.25,
}

def _calculate_attempt_score(cycle_summary):
    coverage_ratio = cycle_summary.get('coverage_ratio', 0.0)
    complete_ratio = cycle_summary.get('complete_ratio', 0.0)
    fleury_ratio = cycle_summary.get('fleury_ratio', 0.0)

    weighted_ratio = (
        coverage_ratio * SCORE_WEIGHTS['edge_coverage']
        + complete_ratio * SCORE_WEIGHTS['cycle_complete']
        + fleury_ratio * SCORE_WEIGHTS['fleury_rule']
    )

    attempt_score = int(TEST_TOTAL_SCORE * weighted_ratio)
    return max(0, attempt_score)
```

Приложение В

Материалы этапа интеграции с LMS

Для локальной разработки и интеграции в LMS были подготовлены разные конфигурации запуска. В листинге 4 показан сокращённый фрагмент локальной конфигурации.

Listing 4: Фрагмент локальной конфигурации запуска

```
services:
  task:
    build:
      context: ./src
    ports:
      − ”3000:80”
    environment:
      − APP_MODE=${APP_MODE:−training}
      − TEST_MAX_ATTEMPTS=3
      − TEST_TOTAL_SCORE=5
      − TEST_MIN_SCORE=3
      − TEST_MAX_SCORE=5
      − TEST_TIME_LIMIT_MINUTES=60
      − DEBUG=1
```

В production-конфигурации, напротив, задаются лимиты ресурсов, параметры сетевого окружения и переменные, необходимые для взаимодействия с внешней инфраструктурой системы дистанционного обучения.

Приложение Г

Материалы этапа тестирования

В таблице 5 приведены характерные ситуации, проверявшиеся на этапе тестирования, и результаты выполненных исправлений.

Таблица 5 – Характерные результаты этапа тестирования

Проверяемая ситуация	**Результат и выполненная доработка**
Пустой или неэйлеров граф	Добавлена явная проверка корректности графа до запуска алгоритма, пользователю выводится объясняющее сообщение.
Пошаговое выполнение алгоритма	Уточнена синхронизация состояния между сервером и интерфейсом: журнал, маршрут и подсветка обновляются согласованно.
Определение мостов и вынужденных ходов	Скорректирована логика отображения шага, чтобы в интерфейсе различались обычный выбор ребра и вынужденный переход.
Контролируемое тестирование	Проверены ограничения по времени, числу попыток и формату ввода; некорректные ответы обрабатываются без аварийных состояний.
Повторный контейнерный запуск	Подтверждено, что приложение стартует через подготовленные конфигурации без ручной правки исходного кода.

Таким образом, в приложениях зафиксированы материалы этапов 2–5: результаты согласования требований и проектирования, ключевые фрагменты реализации, конфигурация интеграции с LMS и итоги тестирования с выполненными исправлениями.